L U X O R A

Hotel Booking Management System

A full-stack web application for booking hotel rooms, taking payments and managing a hotel.

PROJECT TYPE	Full-stack web application
FRONTEND	React 19, Vite, React Router, Axios
BACKEND	Java 21, Spring Boot 4, Spring Data JPA
DATABASE	MySQL 8
SUBMITTED BY	_____
GUIDE	_____
DATE	September 2026

CONTENTS

1. Introduction and objectives
2. Technology stack
3. System architecture
4. Modules and features
5. Database design
6. REST API
7. Business rules and validation
8. Security
9. Frontend and user flow
10. Screenshots
11. Testing
12. Installation and setup
13. Limitations and future scope
14. Conclusion

CHAPTER 1

Introduction and Objectives

1.1 Introduction

Luxora is a hotel booking management system. Guests use it to browse rooms, book a stay for chosen dates, pay for the booking and manage their reservations. Hotel staff with an administrator account use a dashboard to manage rooms and to see every booking, guest and payment.

The application is built as a **three-tier web application**: a React single-page frontend running in the browser, a Spring Boot REST API on the server, and a MySQL relational database. The two parts communicate using JSON over HTTP.

1.2 Problem statement

Small hotels often handle bookings by phone, paper registers or spreadsheets. This leads to double bookings, pricing mistakes and no single view of occupancy or revenue. Guests cannot see available rooms or book at any time of day.

1.3 Objectives

1. Let guests register, log in, browse rooms and book a stay online.
2. Prevent a room from being booked twice for overlapping dates.
3. Calculate prices and payment amounts on the server so they cannot be tampered with.
4. Let guests pay for a booking and view, pay for or cancel their bookings.
5. Give administrators a dashboard to add, edit, enable, disable and delete rooms, and to view and cancel bookings.
6. Store passwords securely and restrict admin features to admin users.
7. Provide a clean, responsive interface that works on desktop and mobile.

1.4 Scope

The system covers the complete guest booking journey (search → room details → booking → payment → my bookings) and the administrator's room and booking management. Payment is **simulated**: the system records the payment method and amount, but does not connect to a real payment gateway.

CHAPTER 2

Technology Stack

LAYER	TECHNOLOGY	PURPOSE
User interface	React 19	Component-based single-page application; state handled with `useState` and `useEffect` hooks.
Build tool	Vite	Fast development server and production build.
Routing	React Router 7	Maps URLs such as `/rooms/:id` to page components.
HTTP client	Axios	Sends requests from the browser to the REST API.
Styling	Plain CSS	One stylesheet per page or component, with responsive media queries.
Backend language	Java 21	Server-side business logic.
Framework	Spring Boot 4	REST controllers, dependency injection, embedded web server.
Persistence	Spring Data JPA / Hibernate	Maps Java entity classes to database tables; generates queries.
Database	MySQL 8	Relational storage for users, rooms, bookings and payments.
Security	spring-security-crypto (BCrypt)	One-way hashing of user passwords.
Build (backend)	Maven	Dependency management and build.
Version control	Git and GitHub	Source history and hosting.

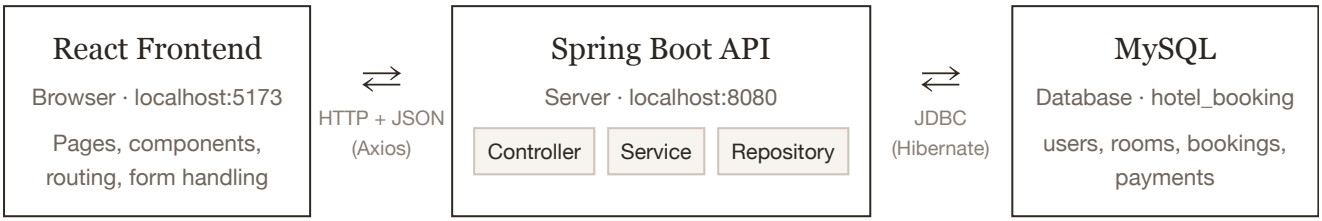
Why these technologies

- **React** lets the interface update instantly (for example the live price preview while choosing dates) without reloading the page.
- **Spring Boot** provides a production-grade web server and dependency injection with very little configuration.
- **Spring Data JPA** removes most hand-written SQL: repository methods such as `findByUserId` are turned into queries automatically.
- **MySQL** enforces the relationships between tables with foreign keys, which suits bookings that must always point to a valid user and room.

CHAPTER 3

System Architecture

The system follows a **three-tier client–server architecture**.



3.1 Backend layers

LAYER	CLASSES	RESPONSIBILITY
Entity	User, Room, Booking, Payment	Java classes mapped to database tables with JPA annotations (`@Entity` , `@ManyToOne` , `@OneToOne`).
Repository	UserRepository, RoomRepository, BookingRepository, PaymentRepository	Database access. Extend `JpaRepository` , which gives save, find, delete and custom queries.
Service	UserService, RoomService, BookingService, PaymentService	All business rules: validation, price calculation, double-booking check, password hashing, admin check.
Controller	UserController, RoomController, BookingController, PaymentController	Receive HTTP requests, read the body and headers, call the service, return JSON.

3.2 Request flow — creating a booking

1. The guest submits the booking form. React sends `POST /api/bookings` with the user, room, dates and guests.
2. `BookingController` receives the request and passes the booking to `BookingService` .
3. `BookingService` validates the dates, loads the room, checks availability, capacity and overlapping bookings, then calculates the price.
4. `BookingRepository` saves the booking; Hibernate issues the SQL `INSERT` .
5. The saved booking is returned as JSON and React navigates to the payment page. If any rule fails, the API returns an error code and message that React shows to the guest.

3.3 Design principles

- **Separation of concerns** — HTTP handling, business rules and data access live in separate layers.
- **Dependency injection** — services and repositories are supplied to each class through its constructor by Spring.
- **Server is the source of truth** — prices, statuses and roles are decided by the server, never trusted from the browser.

CHAPTER 4

Modules and Features

User module

- Register with name, email and password
- Duplicate emails rejected
- Log in; wrong credentials rejected
- Navbar changes after login and logout
- New accounts always get the `USER` role

Room module

- Browse all rooms with photo, capacity and price
- Filter rooms by number of guests
- Room details page
- Featured rooms on the home page, loaded from the database
- Unavailable rooms marked with a badge

Booking module

- Pick check-in, check-out and guests
- Live price preview (nights × price)
- Server validation and double-booking prevention
- My Bookings page with status and payment
- Cancel a confirmed booking

Payment module

- Pay by UPI, card or cash (simulated)
- Amount taken from the booking by the server
- A booking can be paid only once
- Cancelled bookings cannot be paid
- Payment success screen; "PAID" shown in My Bookings

Admin module

- Dashboard: total rooms, confirmed bookings, registered users, revenue
- Rooms tab: add and edit rooms in a form
- Enable or disable a room with one click
- Delete a room (blocked if it has bookings)
- Bookings tab: guest, room, dates, total, payment method, status
- Filter bookings by status; newest first
- Cancel a confirmed booking
- Visible only to admins; "Admin" link in the navbar

Hotel inventory

The system is loaded with 10 rooms across three floors:

ROOM	TYPE	GUESTS	PRICE
101	Luxury Deluxe	2	₹3,000
102	Standard Room	2	₹2,000
103	Deluxe Room	2	₹3,000
104	Luxury Suite	4	₹5,000
105	Classic Room	2	₹2,500

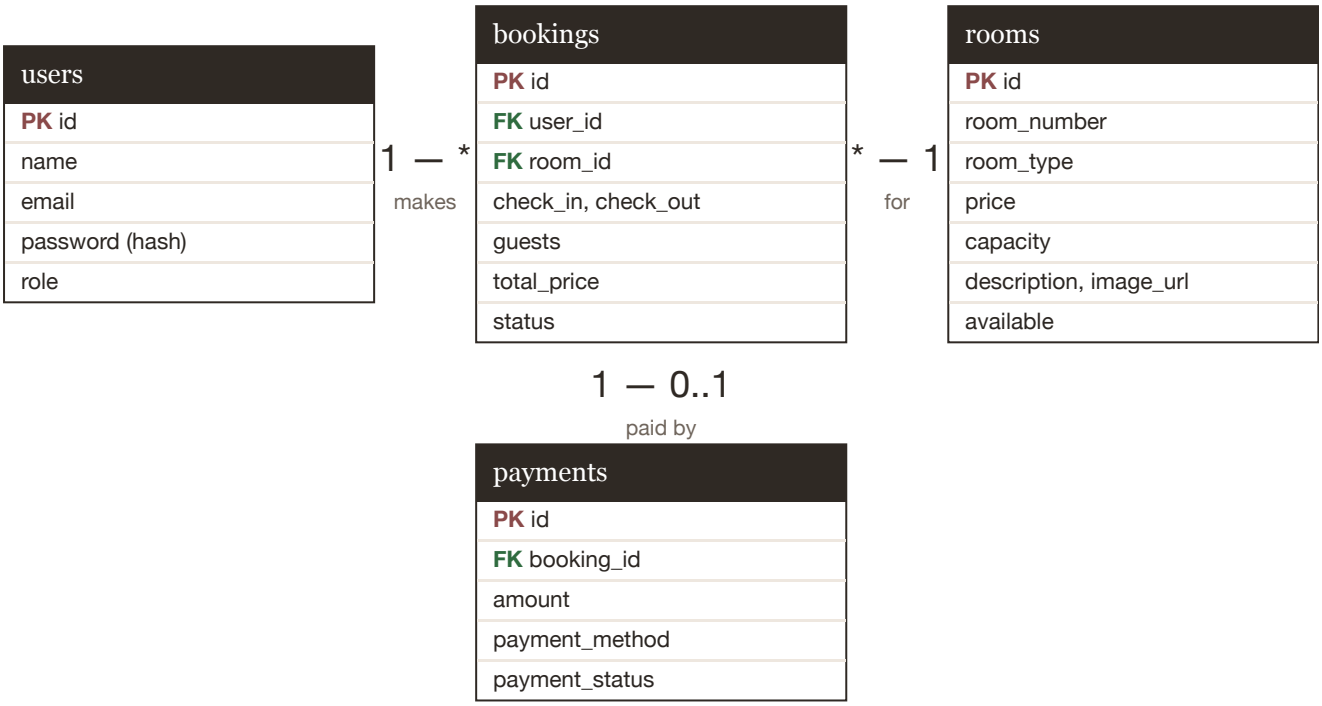
ROOM	TYPE	GUESTS	PRICE
201	Executive Room	2	₹4,000
202	Premium King Room	2	₹3,500
203	Family Room	4	₹4,500
301	Grand Suite	3	₹6,500
302	Presidential Suite	4	₹9,000

CHAPTER 5

Database Design

The database `hotel_booking` has four tables. Hibernate creates and updates them from the entity classes (`spring.jpa.hibernate.ddl-auto=update`).

5.1 Entity–relationship diagram



5.2 Relationships

RELATIONSHIP	TYPE	JPA MAPPING
User → Bookings	One-to-many	`@ManyToOne` on `Booking.user`
Room → Bookings	One-to-many	`@ManyToOne` on `Booking.room`
Booking → Payment	One-to-one (optional)	`@OneToOne` on `Payment.booking`

5.3 Tables

TABLE	COLUMNS AND ALLOWED VALUES
users	id, name, email, password (BCrypt hash, never plain text), role: `USER` or `ADMIN`
rooms	id, room_number, room_type, price (per night), capacity (max guests), description, image_url, available (true/false)
bookings	id, user_id, room_id, check_in, check_out, guests, total_price (set by server), status: `CONFIRMED` or `CANCELLED`
payments	id, booking_id, amount (set by server), payment_method: `UPI`, `CARD` or `CASH`, payment_status: `SUCCESS`

CHAPTER 6

REST API

All endpoints are under `http://localhost:8080/api`. `ADMIN` endpoints require the header `X-User-Id` to identify an administrator. Every error returns a JSON body with a readable `message`, such as "*This room is already booked for the selected dates.*", which the frontend shows to the user.

METHOD	ENDPOINT	DESCRIPTION	ACCESS
POST	`/users/register`	Create an account (role always USER)	PUBLIC
POST	`/users/login`	Log in; 401 on wrong credentials	PUBLIC
GET	`/users`	List all users	ADMIN
GET	`/users/{id}`	Get one user	PUBLIC
GET	`/rooms`	List rooms	PUBLIC
GET	`/rooms/{id}`	Get one room	PUBLIC
POST	`/rooms`	Add a room	ADMIN
PUT	`/rooms/{id}`	Update a room (404 if missing)	ADMIN
DELETE	`/rooms/{id}`	Delete a room without bookings	ADMIN
POST	`/bookings`	Create a validated booking	PUBLIC
GET	`/bookings`	List all bookings	ADMIN
GET	`/bookings/{id}`	Get one booking	PUBLIC
GET	`/bookings/user/{userId}`	A guest's bookings	PUBLIC
PUT	`/bookings/{id}/cancel`	Cancel a booking	PUBLIC
POST	`/payments`	Pay for a booking	PUBLIC
GET	`/payments`	List all payments	ADMIN
GET	`/payments/{id}`	Get one payment	PUBLIC
GET	`/payments/booking/{bookingId}`	Payment for a booking	PUBLIC

HTTP status codes used

CODE	MEANING IN THIS APPLICATION
200 OK	Request succeeded.
400 Bad Request	Invalid data: bad dates, too many guests, room unavailable, unsupported payment method.
401 Unauthorized	Not logged in, or wrong email or password.
403 Forbidden	Logged in, but not an administrator.
404 Not Found	Room or booking does not exist.
409 Conflict	Clashes with existing data: overlapping booking, duplicate email, already paid, room has bookings.

CHAPTER 7

Business Rules and Validation

All rules are enforced on the server in the service layer, so they apply no matter how the API is called.

7.1 Booking rules

#	RULE	RESPONSE
1	Check-out must be after check-in	400
2	Check-in cannot be in the past	400
3	The room must exist	404
4	The room must be marked available	400
5	Guests must be between 1 and the room's capacity	400
6	No overlapping, non-cancelled booking for the same room	409
7	Price = nights × room price; status = CONFIRMED (set by server)	—

7.2 Preventing double booking

Two stays overlap when the existing booking **starts before** the new one ends **and ends after** the new one starts. Cancelled bookings are ignored. This allows a new guest to check in on the same day the previous guest checks out.

```
@Query("SELECT COUNT(b) > 0 FROM Booking b " +
       "WHERE b.room.id = :roomId " +
       "AND b.status <> 'CANCELLED' " +
       "AND b.checkIn < :checkOut " +
       "AND b.checkOut > :checkIn")
boolean existsOverlappingBooking(Long roomId, LocalDate checkIn, LocalDate checkOut);
```

EXISTING BOOKING	NEW REQUEST	RESULT
10 – 12 March	11 – 13 March	Rejected (overlap)
10 – 12 March	12 – 14 March	Allowed (back-to-back)
10 – 12 March (cancelled)	10 – 12 March	Allowed

7.3 Server-side price calculation

```
long nights = ChronoUnit.DAYS.between(checkIn, checkOut);
booking.setRoom(room);
booking.setTotalPrice(nights * room.getPrice());
booking.setStatus("CONFIRMED");
```

Any price or status sent by the browser is overwritten, so a user who edits the request cannot book a room for less.

7.4 Payment rules

- The booking must exist and be `CONFIRMED` (cancelled bookings cannot be paid).
- A booking can be paid only once (409 on a second attempt).
- Payment method must be UPI, CARD or CASH.
- The amount is copied from the booking's total price by the server.

7.5 Room rules

- A room that has bookings cannot be deleted, because that would break booking history. The admin marks it unavailable instead (a **soft delete**).
- Updating a room that does not exist returns 404.

CHAPTER 8

Security

8.1 Password hashing with BCrypt

Passwords are never stored in plain text. On registration the password is hashed with BCrypt; on login the typed password is hashed the same way and compared with the stored hash.

```
user.setPassword(passwordEncoder.encode(user.getPassword()));   // register
passwordEncoder.matches(password, storedPassword);              // login
```

BCrypt adds a random **salt** to every password, so two users with the same password get different hashes, and it is deliberately slow, which makes guessing attacks impractical. Accounts created before hashing was introduced are upgraded to a hash automatically at their next login.

8.2 Passwords never leave the server

```
@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
private String password;
```

The password is accepted in requests but is left out of every JSON response — login, user lists and the user inside each booking.

8.3 Roles and admin access

- New accounts are always given the `USER` role, whatever the browser sends. Administrators are promoted in the database.
- Admin actions send the user's id in the `X-User-Id` header. `UserService.requireAdmin()` loads that user and returns **401** if not logged in or **403** if not an admin.
- Protected actions: adding, editing and deleting rooms; listing all users, bookings and payments.
- The frontend hides the Admin link and shows "Admins only" to other users, but the real protection is the backend check.

8.4 Other measures

- **CORS** — the API only accepts browser requests from the frontend origin `http://localhost:5173`.
- **Server-side validation** — prices, statuses, amounts and roles are decided by the server.
- **Secrets** — the real database password lives in `application.properties`, which is excluded from Git; an example file with a placeholder is committed instead.

Known limitation. The `X-User-Id` header is a simplified form of authentication: a client that knows an administrator's id could send it. A production system would use **Spring Security with JWT** — the server signs a token at login and verifies it on every request, so it cannot be forged.

CHAPTER 9

Frontend and User Flow

9.1 Pages

ROUTE	PAGE	WHAT IT DOES
`/`	Home	Hero banner, search box (dates and guests), three featured rooms from the database.
`/rooms`	Rooms	All rooms as cards; optional filter by number of guests.
`/rooms/:id`	Room details	Full description, capacity, price and a Book Now button.
`/booking/:id`	Booking	Date and guest selection, live price preview, confirm booking.
`/payment/:bookingId`	Payment	Booking summary, payment method, pay; success screen.
`/my-bookings`	My Bookings	Guest's bookings with status and payment; Pay Now and Cancel.
`/login`, `/register`	Login, Register	Account forms with server error messages.
`/admin`	Admin	Dashboard, rooms management and bookings management.

9.2 Reusable components

Navbar (changes with login state and role), **Hero**, **BookingSearch**, **FeaturedRooms**, **RoomCard** (used on the home and rooms pages) and **Footer**.

9.3 Guest journey

Home → search or Explore Rooms → Rooms → Room details → Book Now → choose dates → Confirm Booking → Payment → Payment successful → My Bookings (shows ✓ PAID).

9.4 Admin journey

Log in as admin → Admin link → Dashboard numbers → Rooms tab (add, edit, enable/disable, delete) → Bookings tab (filter, view payment, cancel).

9.5 Key React techniques

- **useState / useEffect** — store page data and load it from the API when a page opens.
- **Promise.all** — the admin dashboard loads rooms, bookings, users and payments in parallel.
- **useParams / useNavigate / useSearchParams** — read ids from the URL, move between pages, read the guest filter.
- **Conditional rendering** — loading spinners, empty states ("No bookings yet"), error messages and role-based links.
- **localStorage** — keeps the logged-in user between page loads; removed on logout.
- **Responsive CSS** — layouts switch to one column below 900 px and 600 px screen widths.

CHAPTER 10

Screenshots

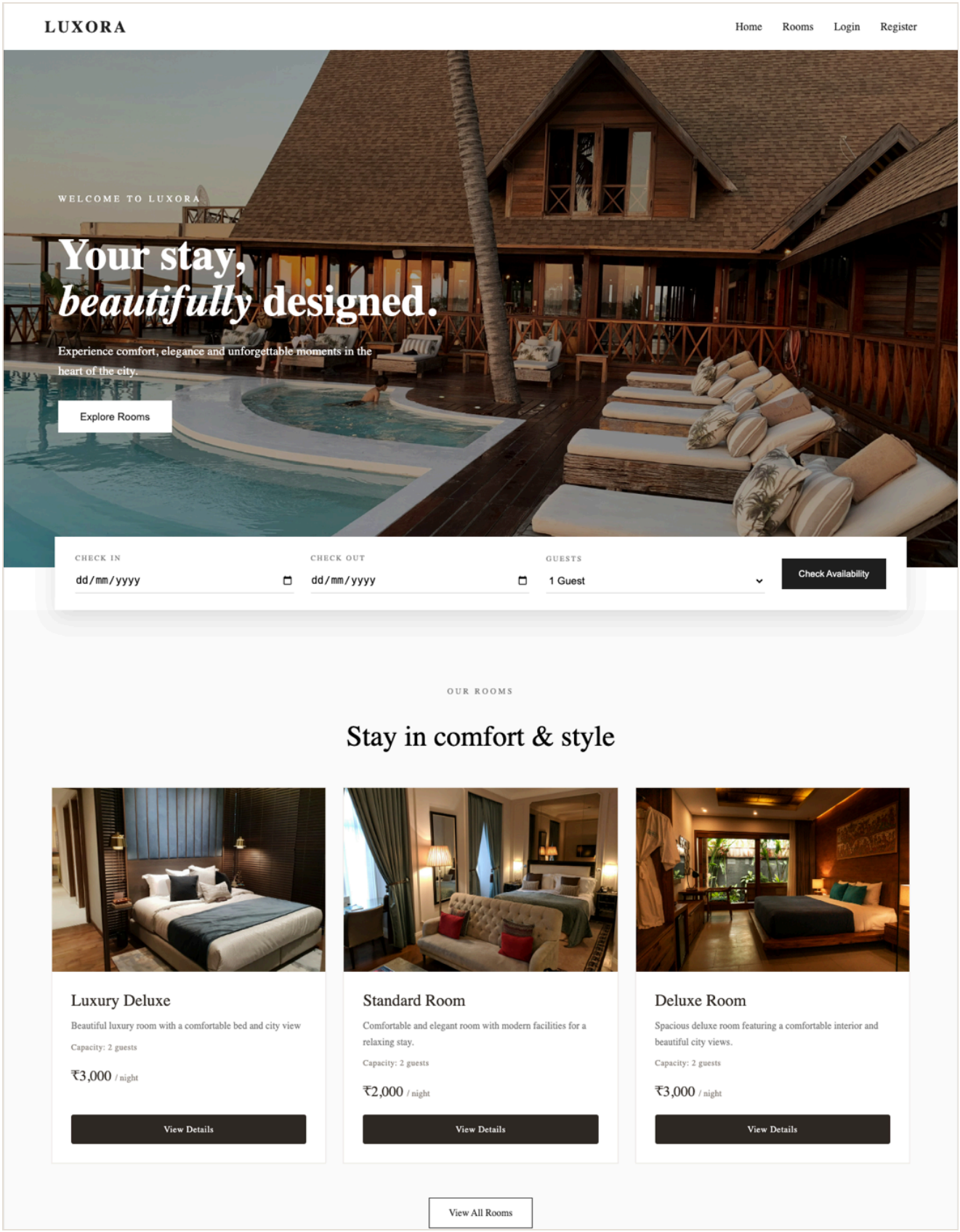


Figure 1 — Home page with hero, search box and featured rooms loaded from the database

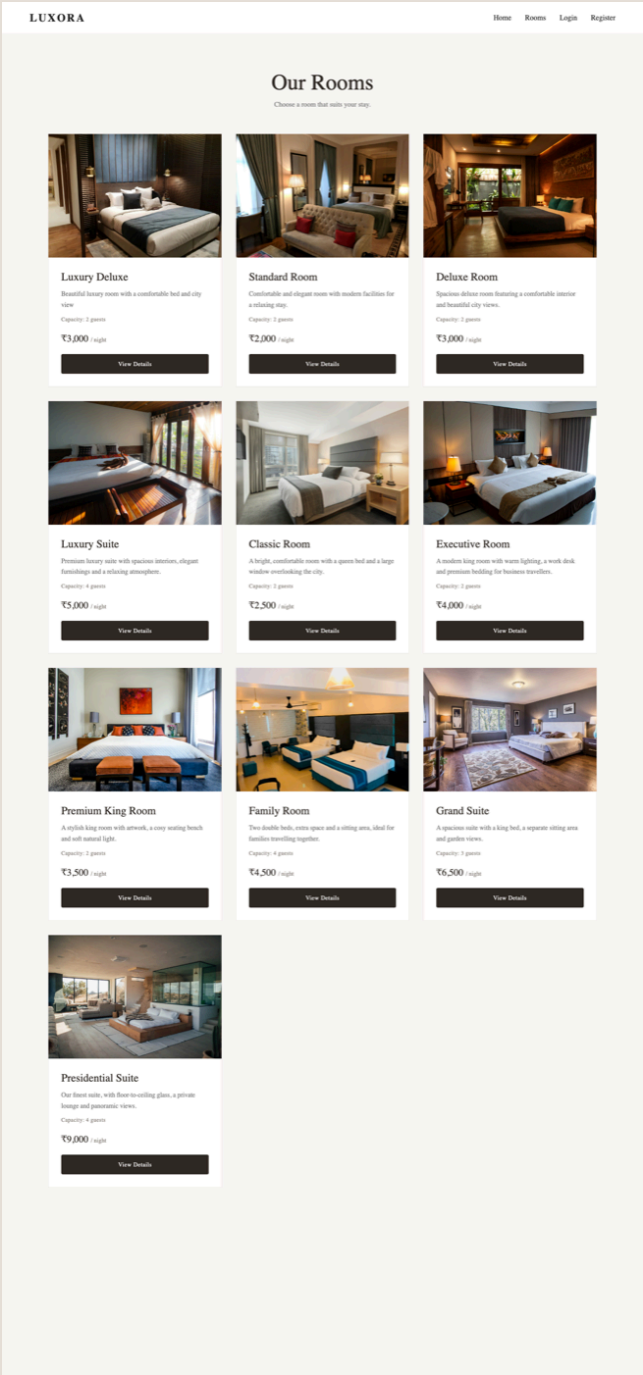


Figure 2 — Rooms page showing all 10 rooms

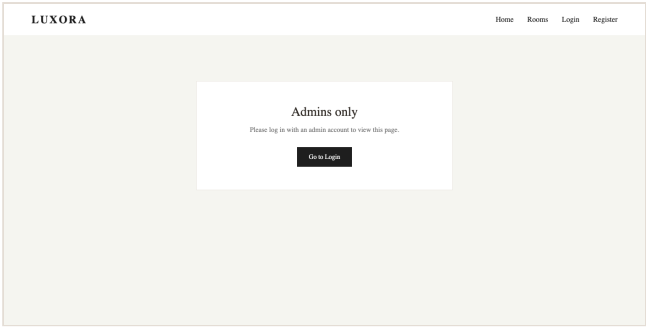


Figure 3 — The Admin page as seen by a visitor who is not an administrator

Screenshots of the Booking, Payment, My Bookings and Admin dashboard pages can be added here from the running application.

CHAPTER 11

Testing

The running application was tested end to end through its REST API (the interface the React frontend uses). Each test sent a real request and checked the status code and response.

#	TEST CASE	EXPECTED RESULT	RESULT
1	List users without logging in	401 "Please log in."	**Pass**
2	List users as a normal user	403 Forbidden	**Pass**
3	Register while sending role ADMIN	Account created with role USER	**Pass**
4	Register with an existing email	409 Conflict	**Pass**
5	Log in with a wrong password	401 Unauthorized	**Pass**
6	Log in with the correct password	User returned, no password field	**Pass**
7	Add a room as a normal user	403 Forbidden	**Pass**
8	Add a room as admin	Room created	**Pass**
9	Book 2 nights, sending price ₹1 and status CANCELLED	Saved as ₹2,000 and CONFIRMED	**Pass**
10	Overlapping booking on the same room	409 Conflict	**Pass**
11	Back-to-back booking (check-in on previous check-out day)	Booking created	**Pass**
12	3 guests in a 2-person room	400 Bad Request	**Pass**
13	Check-in date in the past	400 Bad Request	**Pass**
14	Pay, sending amount ₹1	Saved as ₹2,000, SUCCESS	**Pass**
15	Pay the same booking again	409 Conflict	**Pass**
16	Pay with an unsupported method	400 Bad Request	**Pass**
17	Delete a room that has bookings	409 Conflict	**Pass**
18	Cancel a booking	Status CANCELLED	**Pass**
19	Pay a cancelled booking	400 Bad Request	**Pass**
20	Disable a room as admin	available = false	**Pass**
21	Book a disabled room	400 Bad Request	**Pass**
22	Delete a room without bookings	Room deleted	**Pass**
23	Update a room that does not exist	404 Not Found	**Pass**
24	List users, bookings and payments as admin	200 OK	**Pass**
25	Search admin user list for password fields	None present	**Pass**
26	Check role of newly registered user	USER	**Pass**

Browser checks: the home page loads featured rooms from the database, the Rooms page filters by guest count, and the Admin page shows "Admins only" to non-administrators.

Bug found and fixed: error messages were missing from responses because Spring Boot 4 renamed `server.error.include-message` to `spring.web.error.include-message`.

CHAPTER 12

Installation and Setup

Requirements: Java 21, Node.js 20.19+ (or 22.12+), MySQL 8.

1. **Create the database**

```
CREATE DATABASE hotel_booking;
```

2. **Configure the backend** — copy `application-example.properties` to `application.properties` in `backend/hotelbooking/src/main/resources` and enter the MySQL password.
3. **Start the backend** (creates the tables; runs on port 8080)

```
cd backend/hotelbooking
./mvnw spring-boot:run
```

4. **Load the rooms** — run `database/seed.sql` once in MySQL Workbench.
5. **Start the frontend** (runs on port 5173)

```
cd frontend
npm install
npm run dev
```

6. **Create an administrator** — register in the app, then:

```
SELECT id, email, role FROM users;
UPDATE users SET role = 'ADMIN' WHERE id = <id>;
```

   Log out and log in again to see the Admin link.

Project structure

```
Hotel management System/
├── backend/hotelbooking/        Spring Boot API
│   └── src/main/java/com/hotel/hotelbooking/
│       ├── entity/              User, Room, Booking, Payment
│       ├── repository/          Spring Data JPA repositories
│       ├── service/             Business rules and validation
│       └── controller/          REST endpoints
├── frontend/                    React application
│   └── src/
│       ├── components/          Navbar, Hero, BookingSearch, FeaturedRooms,
│       │                        RoomCard, Footer
│       └── pages/               Home, Rooms, RoomDetails, Booking, Payment,
│                                MyBookings, Login, Register, Admin
├── database/seed.sql            The 10 hotel rooms
└── README.md
```

CHAPTER 13

Limitations and Future Scope

Current limitations

- **Simplified authentication** — the logged-in user is kept in the browser's localStorage and admin requests are identified by the `X-User-Id` header, which a client could fake.
- **Simulated payments** — no real money is taken.
- **No refunds** — cancelling a paid booking does not refund the payment.
- **Search by date** — the Rooms page filters by guests only; date availability is checked when booking.
- **Configuration** — the API address is written directly in the frontend code.

Future enhancements

- Spring Security with JWT tokens and role-based endpoint protection.
- Integration with a payment gateway such as Razorpay or Stripe, with refunds on cancellation.
- Show only rooms free for the selected dates on the Rooms page.
- Email confirmation of bookings and payments.
- Room image upload instead of image links; multiple photos per room.
- Reviews and ratings; discount codes; seasonal pricing.
- Occupancy and revenue charts on the admin dashboard; automated unit and integration tests.
- Deployment to the cloud with an environment-based API URL.

CHAPTER 14

Conclusion

Luxora delivers a complete hotel booking workflow: guests can find a room, book it for specific dates, pay and manage their bookings, and administrators can manage rooms and oversee all bookings, users and revenue from one dashboard.

The project applies a layered Spring Boot architecture with JPA entities and relationships, a responsive React frontend, and server-side business rules that prevent double bookings and price tampering. Passwords are protected with BCrypt and administrative features are restricted by role. All 26 end-to-end test cases passed.

The limitations identified — chiefly token-based authentication and real payment processing — form a clear path for future development.